

Resumo - Semana 4 do Toytrace

Ordem dos arquivos e funções a implementar

Arquivo principal da Semana 4

A implementação fica concentrada em `src/trace_runtime.c`. A Semana 3 já faz o processo parar em syscalls; agora a Semana 4 lê os registradores e preenche `struct syscall_event`.

Ordem de implementação

#	Arquivo / função	O que implementar	Resultado esperado
1	<code>src/trace_runtime.c</code> <code>trace_program()</code>	No loop, após <code>wait_for_syscall_stop()</code> , chamar <code>ptrace(PTRACE_GETREGS)</code> para preencher regs.	Registradores lidos; em erro, retornar -1.
2	<code>src/trace_runtime.c</code> <code>fill_event_from_regs()</code>	Preencher <code>pid</code> , <code>entering</code> , <code>syscall_no</code> , <code>ret</code> e <code>args[0..5]</code> .	Evento cru completo.
3	<code>src/student/formatter.c</code> <code>student_debug_raw_event()</code>	Usar só como apoio no modo <code>--raw-events</code> . Não é a formatação final.	Conferir <code>pid</code> , <code>syscall</code> e entrada/saída.

Mapa rápido dos registradores

Campo	Registrador	Observação
<code>syscall_no</code>	<code>orig_rax</code>	Número da syscall.
<code>ret</code>	<code>rax</code>	Valor válido na saída.
<code>args[0..5]</code>	<code>rdi</code> , <code>rsi</code> , <code>rdx</code> , <code>r10</code> , <code>r8</code> , <code>r9</code>	Argumentos capturados na entrada.

Como os arquivos/funções conversam

```

trace_program()
-> wait_for_syscall_stop()      [Semana 3]
-> PTRACE_GETREGS              [Semana 4]
-> fill_event_from_regs()      [Semana 4]
-> observer(&ev, userdata)     [Semana 4]
    -> main.c: raw-events ou pairer
-> resume_until_next_syscall() [Semana 3]

```

Checklist - Semana 4

- Implementar somente **PTRACE_GETREGS** e **fill_event_from_regs()** em `src/trace_runtime.c`.
- Não mexer em **pairer.c**: ficou na Semana 2.
- Não fazer a formatação final: **formatter.c** fica para a Semana 5.

Comandos de teste

```

make
./toytrace trace --raw-events -- ./tests/targets/hello_write
./toytrace trace --raw-events -- /bin/echo oi

```

Resultado esperado: eventos crus com `pid`, `syscall` e indicação de entrada/saída. A impressão final legível fica para a Semana 5.

Resumo final

Na Semana 4, a ordem correta é: ler registradores com `PTRACE_GETREGS`, preencher `struct syscall_event` em `fill_event_from_regs()` e enviar o evento para o observer.